

Z-ANT: Static Memory Planning and Compiler-Level Contributions to an Open-Source Embedded Inference Framework

Matteo Roda and Elia Sandrini

Politecnico di Milano, B.Sc. Engineering of Computing Systems

Abstract — Deploying deep neural networks on microcontroller-class hardware exposes a fundamental mismatch: inference frameworks assume dynamic allocation, runtime graph traversal, and OS services that bare-metal targets cannot provide. Z-ANT resolves this at compile time, producing a dependency-free static library in Zig and C for direct embedded deployment. This work contributes five extensions to Z-ANT: a structured positional error-return scheme for the inference entry point; correctness fixes and ONNX realignment for the `split` and `abs` operators; a redesigned static memory planning subsystem with a configurable offline interval-packing heuristic and an exact branch-and-bound planner for small graphs; a rebuilt continuous-integration pipeline with matrix-based multi-flag testing; and a scoped *CMSIS-NN* re-integration path for Cortex-M quantized convolution. The redesigned planner reduces allocation overhead by 24.8–45.3 percentage points across four of five benchmark models; `resnet50` marks the structural limit of ordering-based planning, where skip-connection peak-pressure tensors saturate the allocation floor.

1. INTRODUCTION

Microcontroller-class hardware and modern deep learning inference frameworks share a common objective—executing neural networks efficiently—yet operate under assumptions that are, at present, structurally incompatible. Inference frameworks are designed around dynamic memory allocation, runtime graph traversal, and operating system services, while embedded targets such as *ARM Cortex-M* and *RISC-V* microcontrollers typically offer less than 512 KB of SRAM, no memory management unit, and no provision for runtime heap management in bare-metal firmware. The result is a systematic mismatch: frameworks optimized for flexibility produce binaries that are too large, too unpredictable, and too dependent on runtime infrastructure for deployment on the devices where inference is most needed.

Z-ANT [9] addresses this mismatch by treating neural-network deployment as a compilation problem rather than a runtime one. Starting from a model expressed in the *ONNX* interchange format, the toolchain resolves all scheduling, memory planning, and optimization decisions at compile time—through the multi-stage pipeline described in section 2—producing a deterministic static library that can be linked directly into embedded firmware without external dependencies. *Zig* is adopted as both the implementation language and the output substrate for its compile-time evaluation, deterministic allocator model, and native C interoperability—properties that align directly with the requirements of predictable embedded inference.

This report describes the following contributions to the Z-ANT toolchain. First, the error-reporting contract of the generated inference entry point is improved: in place of a single hardcoded failure code, a structured positional return scheme is introduced that identifies the failing operator by its index in the execution sequence (section 3).

Second, two operator implementations are corrected and extended: the `split` operator is realigned to the *ONNX* type specification and its preallocated-output memory semantics are fixed; the `abs` operator is implemented to the *ONNX* standard with both lean and normal code-generation paths (section 4). Third, and constituting the primary contribution, the static memory planning subsystem is redesigned: an offline lifetime-based heuristic with configurable tensor-ordering strategies replaces the prior local greedy scheme, complemented by an exact branch-and-bound planner for small graphs, together yielding significant reductions in static allocation overhead across a range of representative models (section 5). Fourth, the continuous integration and delivery infrastructure is rebuilt to support matrix-based multi-flag testing and on-demand workflow execution, substantially expanding coverage after a period of deprecation (section 6). Fifth, *CMSIS-NN*-accelerated quantized convolution on Cortex-M targets is identified as the primary direction for future work, with architectural groundwork established and integration ongoing (section 7).

2. BACKGROUND

Z-ANT operates as a multi-stage compilation pipeline that progressively transforms a neural-network model into a deployable embedded inference library. The pipeline, illustrated in fig. 1, takes as input a model encoded in *ONNX* format and advances it through seven stages—pre-processing, internal-representation construction, graph optimization, linearization, memory planning, code generation, and target compilation—resolving all scheduling, memory, and optimization decisions statically so that the final output requires no runtime infrastructure [1].

The pipeline begins with a pre-processing phase in which the raw *ONNX* file is simplified, cleaned of redundant or ambiguous structures, and enriched with explicit

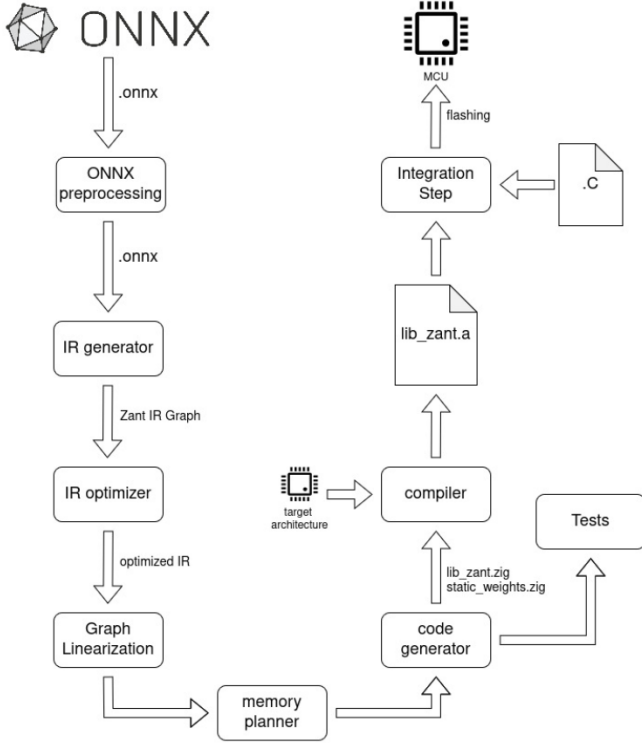


Figure 1. The Z-ANT compilation pipeline, from ONNX input to embedded static library [1].

shape information using *ONNX Runtime*’s shape-inference engine. The normalised, fully shape-resolved graph is then translated into Z-ANT’s *Intermediate Representation* (IR), a deployment-oriented abstraction in which operators, tensors, and their attributes are made explicit to facilitate downstream optimization. Each node in this representation—a *ZantNode*—holds references to its input and output tensors; each tensor—a *ZantTensor*—stores its values as a flat contiguous buffer accompanied by explicit shape metadata. Tensor layouts such as *NCHW* and *NHWC* are semantic interpretations of this flat buffer rather than distinct physical representations: the operator implementation is responsible for applying the correct indexing convention.

Once the IR is constructed, an optimization pass applies three classes of pattern-matching transformation. *Fusion* collapses operator sequences—such as convolution followed by a ReLU activation—into a single composite node. *Substitution* replaces semantically equivalent subgraphs with canonical operators, for instance rewriting a *DequantizeLinear-Conv-QuantizeLinear* sequence as *QLinearConv*. *Elimination* removes complementary node pairs whose composition reduces to an identity. The result is a streamlined graph in which redundant computation and unnecessary quantization boundaries have been excised.

The optimised IR is subsequently linearised through a depth-first traversal that produces a topologically valid execution sequence, with a tie-breaking rule that prioritises nodes closer to their consumers so as to reduce intermediate activation lifetimes. The linearised graph fixes the execution order and constitutes the direct input to the memory planning stage, for which the static variant is

redesigned in section 5.

With execution order established, Z-ANT assigns activation buffers either through a static lifetime analysis—yielding deterministic SRAM usage known at compile time—or via a dynamic allocation strategy for graphs whose memory requirements cannot be bounded statically. The code generator then walks the planned linearised graph and, for each node, invokes the operator’s `write_op` function to emit the corresponding kernel call into the generated `predict()` inference entry point. Static model parameters are placed in read-only memory sections such as `.rodata` or `.qspidata`; intermediate activations occupy `.bss`; and executable code resides in `.text`. Finally, the generated Zig sources are compiled for the target architecture using Zig’s cross-compilation facilities, emitting a fully optimised static library tailored to the selected microcontroller or SoC.

The operator library currently covers over thirty ONNX operators; a full table of supported operations and their implementation status is maintained in the project repository [9]. Correctness of each generated library is validated by feeding identical inputs to both the Z-ANT-generated code and ONNX Runtime, comparing outputs within type-appropriate tolerances.

3. STRUCTURED ERROR REPORTING IN GENERATED INFERENCE CODE

In the original implementation, every core-level runtime failure inside the generated `predict()` function caused it to return the hardcoded value `-1`, irrespective of which operator triggered the failure or at what point in the execution sequence it occurred. This behaviour originated in the `write_op` implementation of each individual operator: the failure return was emitted as a literal with no mechanism to communicate positional context to the caller. Diagnosing inference failures therefore required external instrumentation or manual inspection of the generated code, with no systematic basis for fault localization.

The fix introduces `getMathErrorReturn` in `IR_zant/utils.zig`. The function maintains a counter initialized at 100 and incremented once per operator in the execution sequence; the negated counter value is emitted as the return code on failure, so that `-101` identifies a failure in the first operator, `-102` in the second, and so on. The offset of 100 is deliberate: it reserves the range `[-1,~-99]` for a distinct class of system-level errors, keeping the two error domains non-overlapping without requiring coordination at the call site. Every `write_op` function across the operator library in `IR_zant/op_union` was updated uniformly to call `getMathErrorReturn` in place of the literal `-1`, making the scheme consistent across the entire code-generation layer.

```

1 // before: return literal -1 on core-level error
2 try writer.print(
3     \ \      tensMath.clip_lean(
4 // [...]
5     \ \      ) catch return -1;
6 // after: return getMathErrorReturn() on core-level
   error

```

```

7 try writer.print(
8     \\\tensMath.clip_lean(
9 // [...]
10 \\\) catch return {d};
11 // [...], .{
12 // [...]
13     utils.getMathErrorReturn(),
14 );

```

Listing 1. Representative `write_op` excerpt: hardcoded literal before the change and `getMathErrorReturn()` after.

Systematically traversing all `write_op` implementations to apply the fix surfaced two pre-existing defects, each reported as a separate issue:

- The fused operator `fused_Conv_Sigmoid_Mul` had no codegen path in the IR: the operator was recognized and fused correctly during graph optimization, but the corresponding `write_op` implementation was absent, making it impossible to generate valid inference code for any model containing that pattern.
- The `op_split` operator’s generated call to `tensMath.split_lean` passed a value of type `[]i64` to a parameter typed `[]const usize`, violating the ONNX specification for the `split` input, which mandates a signed 64-bit integer sequence. This defect and its resolution are addressed in the following section (section 4).

4. OPERATOR CORRECTNESS AND IMPLEMENTATION

4.1 ONNX Alignment and Memory Semantics of `split`

The ONNX specification defines the optional `split` input as a sequence of signed 64-bit integers (`tensor(int64)`), encoding the length of each output slice. The corresponding parameter in Z-ANT’s core implementation was typed `[]const usize`—an unsigned platform-width integer—producing a type mismatch that surfaces whenever the generated code passes split sizes to the core. Surfaced during the `write_op` traversal described in section 3, the fix realigns the parameter type to `[]const i64` throughout the operator core, with corresponding updates to the operator logic and its test suite.

A more subtle defect resided in `split_lean`. The function reallocated the `data` and `shape` fields of its output tensors even in contexts where the generated caller had already preallocated those outputs. The generated call site constructs a temporary `Tensor` array to hold the outputs; since array elements in Zig are value types rather than references, the reallocations performed inside `split_lean` affected only these local copies, leaving the originals used and deallocated by the caller entirely unchanged. The result was silent data loss and memory leaks observable only at the level of the complete inference run. The fix moves all allocation responsibility into the non-lean `split` path; `split_lean` is redefined as a strictly preallocated-output routine that validates output shapes and copies data into already-allocated buffers without performing any allocation of its own.

4.2 Element-wise Absolute Value: `abs`

Each operator in Z-ANT exposes two execution paths: a *normal* variant that allocates and returns its own output buffer, and a *lean* variant that writes into a caller-provided pre-allocated buffer instead. This distinction is foundational for generated code, where output buffers are preallocated by the memory planner described in section 5.

The `abs` operator implements the ONNX element-wise absolute value using Zig’s built-in `@abs` intrinsic, iterating over the flat tensor buffer via direct pointer arithmetic to avoid intermediate copies. Both lean and normal execution paths are implemented, and the operator exposes a `write_op` function that emits the appropriate codegen variant for each context. Correctness is established against a Python-generated reference ONNX node: identical inputs are fed to both the Z-ANT implementation and the reference node, with outputs compared element-wise—this script-driven comparison against ONNX Runtime serves as the reference validation method for all operator implementations across Z-ANT.

Both operators described in this section were further validated through unit tests and the single-operation extraction workflow, in which each operator is isolated into its own ONNX subgraph, its generated library compiled, and its outputs exercised independently.

5. STATIC MEMORY PLANNING

5.1 Motivation and Lifetime Model

On SRAM-constrained microcontrollers, dynamic allocation of intermediate tensors risks fragmentation and unrecoverable out-of-memory failures at inference time; Z-ANT offers both dynamic and static memory planning, the latter eliminating these risks by assigning every intermediate tensor to a backing buffer whose size and position are fixed at code-generation time. Allocating one dedicated buffer per tensor would consume memory equal to the sum of all activation sizes—a quantity that routinely exceeds the SRAM of any practical microcontroller—whereas reusing buffers across tensors whose active windows do not overlap reduces that footprint to the irreducible working set. This section presents the complete design of Z-ANT’s static memory planning subsystem: a baseline greedy allocator (section 5.2), an offline interval-packing heuristic (section 5.3), an exact branch-and-bound planner for small graphs (section 5.4), a correctness fix for reused buffers (section 5.5), a practical example comparing the to models (section 5.6), an evaluation across five embedded-inference models (section 5.7), and a reference to the current open directions for the static memory planning system (section 5.8).

The input to the memory planner is the *linearized graph* produced by Z-ANT’s depth-first traversal of the optimized IR—a flat, ordered sequence of N operator nodes, each occupying a unique 0-based integer position called an *execution step*. Every tensor t output by a node is characterized by two step indices: $b(t)$, the execution step of the node that produces t (its *birth* step), and $d(t)$, the execution step of the last node in the sequence that

reads t as one of its inputs (its *death* step). The *liveness* $\ell(t) = d(t) - b(t)$ is the number of steps across which t must remain resident in memory. The death step is determined by scanning all nodes at steps $b(t) + 1$ through $N - 1$ and recording the highest step index at which t appears in a node’s input list; this full-suffix scan guarantees that $d(t)$ is the *last* consumer, not the first. For tensors with no internal consumer—graph outputs being the primary case—the fallback $d(t) = b(t) + 1$ is used; this is deliberately conservative and noted in section 5.8 as a direction for future refinement.

Two tensors t_1 and t_2 of the same element type may share one backing buffer only if their lifetime intervals do not overlap. The planner uses inclusive-bounds semantics throughout: the intervals $[b(t_1), d(t_1)]$ and $[b(t_2), d(t_2)]$ are considered overlapping whenever

$$\neg(d(t_1) < b(t_2) \vee d(t_2) < b(t_1)), \quad (1)$$

so two intervals that share exactly one endpoint are treated as conflicting. The resulting assignment problem—minimizing the total reserved size across a typed set of backing buffers subject to the non-overlap constraint of eq. (1)—combines the structure of typed interval scheduling with that of bin-packing: both problem families, and their typed variants in particular, are NP-hard in the general case [3]. The planners described in sections 5.2 to 5.4 approach this problem with a greedy online baseline, a configurable offline heuristic, and an exact search with pruning, respectively.

5.2 v0: Online Greedy Allocator

The baseline planner, `computeBackingBuffers_v0` in `heuristic_planners.zig`, operates directly on the IR graph rather than on the linearized sequence used by the planners introduced in sections 5.3 and 5.4. A breadth-first pass from the graph root—implemented in `computeNodeEpochsFromStartNode`—assigns each node an *epoch* equal to its maximum edge-depth from the start, yielding the level-ordered queue `nodes_by_epoch` over which the planner iterates. For each output tensor at the current node, a linear scan of a free list of same-type buffers selects the first entry with sufficient capacity (first-fit); if none is found, a new buffer is created sized exactly to that tensor. Buffer reclamation uses reference counting: on assignment, `letChildrenBorrowBufferForTensor` sets the buffer’s count to the number of direct child nodes of the producing node and records the borrow in the per-node `shared_borrows` map. As each node is dequeued from `nodes_by_epoch`, every buffer listed in its borrow record is decremented; any buffer whose count falls to zero is returned immediately to `free_buffers`, making it eligible for the next assignment. The planning phase runs in $O(N \cdot B)$ per dequeue step, where B is the current free-list length, giving $O(N^2)$ worst-case cost across all N nodes.

The planner’s fundamental limitation is its purely local view: at the moment a tensor is assigned, the free list reflects only buffers whose previous tenants have already been reclaimed, with no knowledge of lifetimes not yet encountered—a structural gap addressed by the of-

fline heuristic of section 5.3. `computeBackingBuffers_v0` is retained in `heuristic_planners.zig` for comparison against the improved planners; `use_heuristic_v0` is fixed to `false` at compile time in `codegen.zig`, so the production pipeline never invokes it.

5.3 v1: Offline Interval-Packing Heuristic

The v1 heuristic replaces v0’s online free-list with an offline interval-packing model: rather than assigning buffers as nodes are visited, it first constructs a complete picture of every tensor’s lifetime, then packs intervals into typed buffers globally. The algorithm proceeds in three sequential phases—lifetime collection, heuristic sorting, and best-fit placement—summarized in listing 2 and detailed in the paragraphs below.

```

1 // Phase 1: collect tensor lifetimes
2 infos ← collectTensorInfos(linearized_graph)
3
4 // Phase 2: sort by chosen heuristic ordering
5 sort(infos, tensorInfoLessThan(option))
6
7 // Phase 3: best-fit placement
8 buffers_by_type ← BuffersByType{}
9 for each tensor t in infos:
10     bufs ← getOrCreateBuffersForType(
11         buffers_by_type, t.ty)
12     idx ← findBestPlannedBufferIndex(bufs, t) //
13         smallest compatible
14     if idx = null:
15         bufs.append(PlannedBuffer{ .size = t.size, .
16             ty = t.ty })
17         idx ← |bufs| - 1
18     reserveTensorInterval(bufs[idx], t)
19     result.put(t.name, BackingBuffer(bufs[idx], t))
20 return result

```

Listing 2. Three-phase structure of `computeBackingBuffers_v1` (`heuristic_planners.zig`); pseudocode.

Data structures. Four types defined in `utils.zig` support the three phases. `TensorInfo` records a tensor’s name, element type (`ty`), element count (`size`), `first_step`, `last_step`, and `liveness`; `size` is expressed in elements rather than bytes—the byte footprint is `size × sizeof(ty)`—and `liveness` stores `last_step − first_step` redundantly to avoid recomputation inside the sort comparator. `PlannedBuffer` holds a buffer’s unique id, element-count capacity, element type, and a dynamic list of `AssignedInterval` records, each storing the `first_step` and `last_step` of one tenant tensor. `BuffersByType` maps each `TensorType` to the list of planned buffers of that type; `TensorsBackingBuffers` is a string hash map that records, for every tensor name, the identity and borrow window of its assigned buffer. Phase 1 populates `TensorInfo` entries via `collectTensorInfos`: for each produced tensor at step s , `computeLastStepForTensor` scans the full suffix from $s + 1$ to $N - 1$ without early exit, so that $d(t)$ is the *latest* consumer rather than the first, at a lifetime-collection cost of $O(T \cdot N \cdot I)$, where T is the total number of produced tensors and I is the mean node fan-in.

Sort orderings. Phase 2 sorts the `TensorInfo` list through `tensorInfoLessThan`, whose ordering is selected via the `-Dstatic_planning` build flag; six base modes are available: `size_first`, `pressure_then_size`, `pressure_then_liveness`,

`liveness_first`, `first_step`, and the `enabled` convenience alias (equivalent to `size_first`). Any explicit mode may append the `_inverse_first_step` suffix to reverse the production-step tie-breaker, yielding eleven distinct orderings in total. Because no single ordering dominates across all graph topologies—as the results in section 5.7 confirm—`-Dstatic_planning` was promoted from a `bool` to a string option in `codegen_flags.zig`; accepted values are validated at Zig compiletime, converting unrecognized inputs into compile errors rather than silent runtime mismatches.

All orderings share a common objective: surface the most constrained tensors first, so that the best-fit search reserves dominant slots before smaller or shorter-lived tensors narrow the available packing options.

`size_first` measures constraint by raw element count, on the premise that a buffer created for a large tensor cannot be downsized and therefore represents the most irreversible commitment.

`pressure_then_size` refines this with the *memory-pressure* metric $p(t) = \text{size}(t) \times \ell(t)$: a tensor that is both large and long-lived occupies byte-steps of memory that a short-lived tensor of equal size does not, and ranks ahead accordingly, with `size` as a secondary tie-breaker; this ordering suits graphs where medium-sized but persistently live tensors are as problematic as larger but transient ones.

`pressure_then_liveness` follows the same primary metric but resolves ties by duration rather than size, favouring graphs where tensors are clustered in size and differentiated primarily by liveness.

`liveness_first` ranks by duration alone, which can misfire when the graph contains large but short-lived activations—such as early downsampling tensors—that are placed last and denied buffers already committed to smaller but longer-lived intermediates.

`first_step` applies no constraint reasoning whatsoever, ordering tensors by production step; it serves as a lower-bound comparison, and consistently produces higher total allocation than any constraint-aware mode.

The `_inverse_first_step` suffix reverses the production-step tie-breaker and benefits graphs where later-produced tensors tend to be smaller and offer denser reuse opportunities than earlier-produced ones; where no such correlation exists, the suffix has negligible effect.

Best-fit placement. In Phase 3, `findBestPlannedBufferIndex` scans the `PlannedBuffer` list for the tensor’s type and returns the index of the *smallest* buffer that satisfies `plannedBufferCanHostTensor`—the three-condition check requiring type equality, sufficient element-count capacity, and no `AssignedInterval` in the buffer’s reserved list overlapping $[b(t), d(t)]$ under the inclusive criterion of eq. (1). If no compatible buffer exists, a new `PlannedBuffer` is created at exactly `tensor.size`; the tensor’s interval is then appended via `reserveTensorInterval` and the assignment recorded in `TensorsBackingBuffers`. Placement complexity is $O(T^2 \cdot P)$ in the worst case, where P is the number of

same-type planned buffers: for each of the T tensors the outer loop scans up to P buffers, and each compatibility check iterates over up to T already-assigned intervals.

5.4 Exact Planning by Branch and Bound

The `v1` heuristic carries no optimality guarantee: on certain graph structures it may allocate strictly more memory than the minimum achievable assignment, and no sort-ordering variant can compensate for a fundamentally suboptimal placement. For graphs small enough to make exhaustive search tractable, the pipeline therefore selects an exact planner instead. Automatic selection is handled by `shouldUseBranchAndBound` in `utils.zig`: the exact planner is invoked whenever the linearized graph contains at most 25 nodes, a threshold fixed pragmatically and not tied to any formal complexity analysis. When the condition holds, `codeGenerateFromGraphZant` in `codegen.zig` routes to `computeBackingBuffers_branchAndBound` in `branch_and_bound.zig` in place of `computeBackingBuffers_v1`.

Before the search begins, `computeBackingBuffers_branchAndBound` sorts tensors by the fixed `size_first` ordering; unlike the configurable orderings of `v1`, this choice is structural rather than model-specific: placing the largest—and therefore most constrained—tensors first causes the sole pruning criterion to fire on more branches and earlier in the recursion, since large tensors admit fewer compatible buffers and tighten feasibility constraints sooner. The recursive procedure `branchAndBoundRecursive` then explores the assignment tree: at each level it attempts to place the current tensor into every existing compatible buffer and, separately, into a newly created one, recursing and backtracking after each branch. A branch is pruned immediately whenever the partial solution’s total reserved memory meets or exceeds the current global best; no additional lower-bound estimate is computed. When all tensors have been assigned, the complete solution replaces the global best if its total cost is strictly lower. Worst-case complexity is $O(T!)$ —unavoidable for an exact method—but the pruning criterion reduces the effective search substantially for the small values of T admitted by the threshold, keeping compilation cost negligible within the 25-node bound. The `-Dforce_bnb` flag overrides the threshold and applies the exact planner regardless of graph size; on graphs beyond the threshold the factorial growth of the search space risks prohibitive build-time blowup, so its use outside debugging and cross-comparison is not recommended.

5.5 Buffer Zeroing on Reuse

Buffer reuse introduces a silent correctness hazard: an operator that accumulates into its output inherits its residual values from the buffer’s previous tenant, corrupting inference results with no error signal. The fix, applied in `write_graphSerialization` (`src/codegen/predict/predict.zig`), emits a `@memset(tensor_<name>.data, 0)` into the gener-

ated `predict()` body before each operation; zeroing is suppressed for in-place operators via the Zig `for...else` pattern shown in listing 3, and the cost is paid on every inference call.

```

1 clear_outputs(writer, node):
2     // retrieve the input tensors for the node
3     input_tensors = node.get_input_tensors()
4
5     // loop through all output tensors of the node
6     for each output_tensor in node.get_output_tensors():
7
8         // check against every input tensor
9         for each input_tensor in input_tensors:
10
11             // if the input tensor name matches the
12             // output tensor name
13             if input_tensor.name equals output_tensor.name:
14                 break // exit the inner loop immediately
15
16             // if the inner loop finished completely
17             // without hitting the 'break' statement
18             else:
19                 // get the sanitized name of the output
20                 // tensor
21                 sanitized_name = output_tensor.getNameSanitized()
22
23                 // write the memset string to the writer
24                 // output
25                 print to writer: "@memset(tensor_" +
26                     sanitized_name + ".data, 0);"
```

Listing 3. Simplified pseudocode sketch of `clear_outputs` in `src/codegen/predict/predict.zig`. The `for-else` construct suppresses zeroing when an output tensor is also listed as an input of the same node.

5.6 A Worked Example

Consider the eight tensors depicted in fig. 2: A , B , C , and D , each of size 3, with lifetimes $[0, 1]$, $[0, 2]$, $[1, 3]$, and $[2, 3]$; X and Y , each of size 4, with lifetimes $[4, 6]$ and $[6, 8]$; M , of size 6, with lifetime $[7, 10]$; and L , of size 10, with lifetime $[10, 11]$. All eight tensors have the same element type. The intervals are inclusive: two tensors

whose lifetimes share an endpoint cannot reuse the same backing buffer.

The branch subgraph for A , B , C , and D is:

$$\text{Op}_0 \rightarrow \{\text{Op}_1, \text{Op}_2\} \rightarrow \text{Op}_3,$$

where Op_0 produces A and B , Op_1 reads A and produces C , Op_2 reads B and produces D , and Op_3 reads C and D . The later tensors X , Y , M , and L are produced in chronological order with the lifetimes listed above.

The minimum achievable allocation is 19 elements. A simple lower bound is obtained from L and M : because $[7, 10]$ and $[10, 11]$ overlap at step 10, they require distinct backing buffers of size 6 and size 10. At steps 1 and 2, the tensors A , B , C , and D require three simultaneous size-3 slots across the branch, so a third backing buffer of size 3 is also necessary. Thus any valid plan requires at least $10 + 6 + 3 = 19$ elements, and the `v1` plan below reaches that bound.

v0 on this graph. The `v0` planner allocates each node’s outputs before releasing that node’s input borrows. It also calls `letChildrenBorrowBufferForTensor` for every output tensor and assumes that every child of the producing node reads that output tensor.

Concretely, Op_0 has two children: the node that computes C and the node that computes D . `v0` therefore registers both children as borrowers of both A and B , even though the first child reads only A and the second child reads only B . When Op_1 executes at step 1, the reference count for A ’s buffer drops from 2 to 1; the buffer remains locked because `v0` still believes Op_2 may read A . At step 2, when D is allocated, A ’s size-3 slot is still not on the free list, so D receives a dedicated size-3 slot.

The remaining dedicated slots follow from chronological allocation and the inclusive endpoint model. X and Y cannot share because $[4, 6]$ and $[6, 8]$ overlap at step 6. M is too large for the previous size-3 and size-4 buffers. L

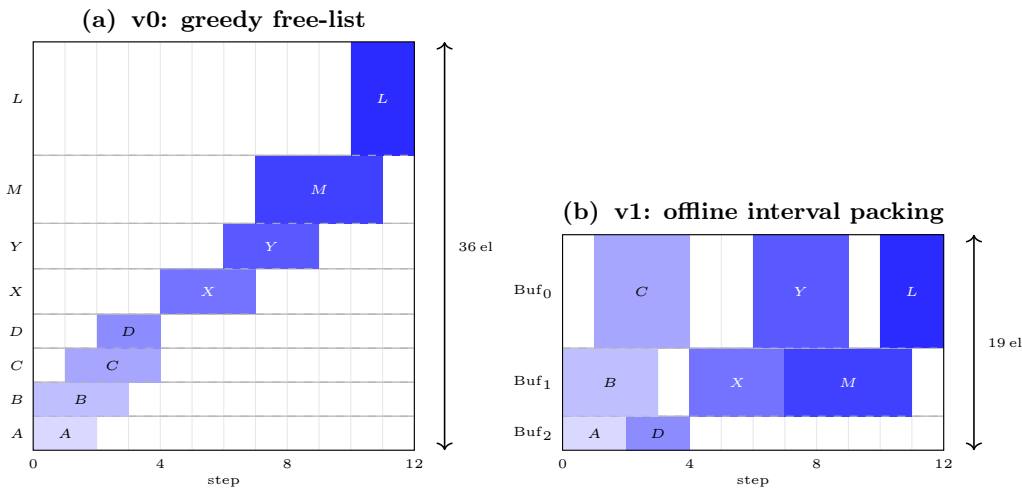


Figure 2. Memory allocation for the eight-tensor worked example. Band height is proportional to element-count capacity. Because lifetimes are inclusive, each lifetime $[b, d]$ is drawn as the half-open display range $[b, d + 1)$, so endpoint conflicts are visible in the plot. White regions are idle periods within each allocated buffer. (a) `v0` misses the A/D reuse because its child-borrow approximation keeps A ’s buffer locked until after D has already been allocated; the later tensors receive dedicated slots because of chronological allocation, endpoint overlap, and size incompatibility. (b) `v1` recovers 17 elements through exact last-use analysis, `size_first` ordering, and best-fit placement: A and D reuse `Buf_2`, X is placed into the tighter 6-element `Buf_1` instead of the 10-element `Buf_0`, and the final allocation is 19 elements total.

is too large for M 's size-6 buffer and also overlaps M at step 10. The resulting v0 allocation is therefore:

$$3 + 3 + 3 + 3 + 4 + 4 + 6 + 10 = 36 \text{ elements.}$$

v1 on this graph. Phase 1 records exact last-use steps from the graph. The pair (A, D) is identifiable as non-overlapping because $d(A) = 1 < 2 = b(D)$, so their lifetimes are disjoint. v0 misses this reuse opportunity because A 's buffer is still locked when D is allocated.

Phase 2 with **size_first** sorts tensors by decreasing size, then breaks equal-size ties by liveness and first step. For this example the order is:

$$[L(10), M(6), X(4), Y(4), B(3), C(3), A(3), D(3)].$$

In Phase 3, L opens Buf_0 of size 10. M cannot share it because $[7, 10]$ and $[10, 11]$ overlap at endpoint 10, so Buf_1 of size 6 is created. For X , both existing buffers are compatible, but best-fit selects Buf_1 because it is the tighter slot: $6 \geq 4$, whereas Buf_0 would reserve a size-10 slot for a size-4 tensor. This preserves Buf_0 for Y , which cannot enter Buf_1 because it overlaps X at step 6 and M from steps 7 to 8.

B also enters Buf_1 by best-fit. C is excluded from Buf_1 by B 's lifetime overlap and joins Buf_0 . A overlaps both B and C , so it opens Buf_2 of size 3. D then shares Buf_2 with A because $d(A) = 1 < 2 = b(D)$. The final v1 allocation is:

$$\text{Buf}_0(10) + \text{Buf}_1(6) + \text{Buf}_2(3) = 19 \text{ elements.}$$

5.7 Results and Tooling

All measurements are assessed compile-time: they reflect the static allocation plan emitted by the code generator, independent of any target hardware or runtime profiling. The overhead ratio $\rho = \text{total_alloc} / \text{peak_live} \times 100\%$ measures distance from the theoretical minimum (a ratio of 100% means total equals peak live); ρ is computed directly from the generated JSON plan for each model. The best **-Dstatic_planning** flag per model was identified by exhaustive enumeration via

compare_static_codegen_flags.py; **mnist-8** is omitted because its graph falls within the 25-node threshold of section 5.4 and is always routed to the branch-and-bound planner regardless of flag.

Analysis. Four of the five models exhibit meaningful overhead reductions, ranging from 24.8 to 45.3 percentage points (table 1). The **mobilenet_v2** model approaches the theoretical optimum at 106.3%, while **new2** benefits most, with ρ dropping from 229.7% to 125.6%. **size_first** is optimal or joint-optimal across the improved models: because backing buffers cannot shrink once opened, placing the largest tensors first creates wide reuse slots that smaller tensors fill whenever lifetimes permit.

Three model-specific properties explain the most notable results:

- **beer.** The graph is a quantized MobileNet-style chain in which many tensors tie on both size and liveness, making the first-step tie-breaker decisive. Reversing it (**size_first_inverse_first_step**) places later equal-sized short tensors into already-open large buckets rather than opening a separate 18,432-byte **u8** slot, reaching 6 buffers and 1,059,840 bytes.
- **mobilenet_v2.** The regular inverted-residual structure concentrates peak pressure on two simultaneous 24,576-element **f32** tensors (196,608 bytes). **size_first** places these two tensors first; their lifetimes do not overlap, so both map to separate same-sized buffers, with two small 1,536-element scratch buffers accounting for the remainder—totalling 208,896 bytes, only 6.25% above the theoretical lower bound.
- **new2.** The graph is quantized and concat-heavy, producing many short-lived **u8** tensors in a small set of recurring sizes. Skip and average-pool branches live only until the next concat and then die; v0 allocates before releasing current inputs, so branch joins force extra same-sized buffers. v1's exact lifetime tracking collapses these recurring size tiers into 7 buffers totalling 138,880 bytes (fig. 3).

Model	Buffers		Total (B)		ρ (%)		Δ (%)	Flag
	v0	v1	v0	v1	v0	v1		
beer	9	6	1 410 048	1 059 840	159.4	119.8	−24.8	size_first_inverse_first_step
beer	9	6	1 410 048	1 161 216	159.4	131.3	−17.6	pressure_then_size
new2	9	7	254 080	138 880	229.7	125.6	−45.3	size_first, pressure_then_size, pressure_then_liveness
new2	9	7	254 080	143 488	229.7	129.7	−43.5	liveness_first, liveness_first_inverse_first_step
new2	9	7	254 080	180 352	229.7	163.1	−29.0	first_step
mobilenet_v2	7	4	323 584	208 896	164.6	106.3	−35.4	size_first, size_first_inverse_first_step
mobilenet_v2	7	5	323 584	210 944	164.6	107.3	−34.8	liveness_first, liveness_first_inverse_first_step
r18_net	6	3	221 184	147 456	168.8	112.5	−33.3	size_first, pressure_then_size
r18_net	6	4	221 184	155 648	168.8	118.8	−29.6	pressure_then_liveness
r18_net	6	4	221 184	163 840	168.8	125.0	−25.9	size_first, pressure_then_liveness
resnet50	3	4	9 633 792	9 633 792	150.0	150.0	0.0	liveness_first, size_first_inverse_first_step
resnet50	3	3	9 633 792	10 035 200	150.0	156.3	+4.2	pressure_then_size, size_first

Table 1. v1 static memory planning result per model. Overhead $\rho = \text{total_alloc} / \text{peak_live} \times 100\%$; Δ is the reduction in ρ relative to v0 in percentage points: $\Delta = \frac{(\rho_{v1} - \rho_{v0})}{\rho_{v0}} \times 100\%$. Tied flags are listed as flag1, flag2.

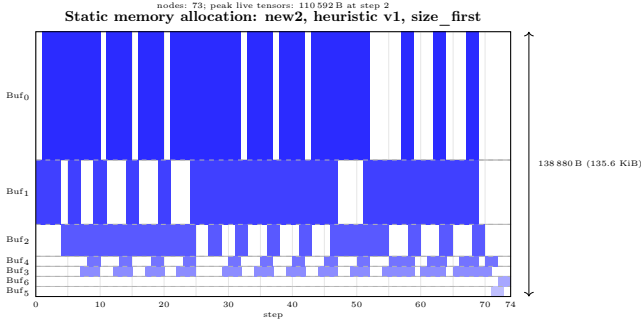


Figure 3. Static memory allocation for `new2`. The stacked bands are planner backing buffers over execution steps: buffers 0–4 store `u8` quantized activations, while buffers 5–6 store `f32` output-path tensors. Because buffers are type-specific, the `f32` tensors cannot reuse the existing `u8` buffers, so separate `f32` buffers are required. Shaded regions show tensor lifetimes; white regions are allocated but idle space.

ResNet50. `resnet50` is the sole model for which `v1` achieves no improvement: the best available result matches `v0` exactly at 9 633 792 B ($\rho = 150.0\%$), while the majority of orderings actively increase allocation—`pressure_then_size` and `size_first` raise the total to 10 035 200 B ($\rho = 156.3\%$, +4.2 pp), worse than `v0`.

The cause is structural: skip connections force large early feature maps to remain live until their corresponding element-wise add nodes, congesting the highest-resolution convergence points before spatial downsampling occurs. The 3 211 264 B gap between peak live memory (6 422 528 B) and total allocation is filled by buffers strictly constrained by these peak-pressure tensors; no reassignment can reduce this residual, forcing both planners to the same floor. While `v1` generates four buffers instead of `v0`’s three, the total memory allocated is completely independent of the buffer count.

The `resnet50` result marks a fundamental limit of ordering-based planning: when congestion-forced tensors dominate the allocation, the improvement ceiling is zero. Mitigating this bottleneck requires graph-transforming optimizations—such as operator fusion to merge convolution and addition steps directly within hardware registers, in-place memory sharing to overwrite residual inputs, or layout transformations that compress active spatial footprints.

Tooling. Three tools accompany the planner. When static planning is active, Z-ANT writes the selected plan to `generated/<model>/memory_allocation_<model>.json`, whose `"metadata"` object records the planner name, `-Dstatic_planning` option, `-Dforce_bnb` flag, and linearized node count, and whose `"tensors"` array maps each tensor name to its backing-buffer assignment. `compare_static_codegen_flags.py` invokes `zig build lib-gen` for every supported ordering and writes an HTML report to `generated/<model>/static_codegen_planner_comparison.html`, with columns for static allocation, overhead, buffer count, planner selection, and build time; the minimum-allocation row is highlighted automatically. `visualise_static_memory_allocation.py` reads the JSON plan and renders an interactive Gantt chart

of tensor lifetimes and buffer assignments, with hover annotations for per-tensor size, buffer identity, and borrow window.

5.8 Open Directions for Static Memory Planning

The static memory planner leaves several implementation directions open for a subsequent continuation of the project. The first concerns the granularity of buffer compatibility. The current planner partitions candidate buffers by tensor element type: as visible in fig. 3, `u8` buffers of sufficient capacity are never assigned to `f32` tensors, preventing reuse across types that differ only in their scalar representation. A future implementation could replace this type-partitioned model with a *common byte-buffer representation*, provided that each assignment preserves the required byte size, alignment, and typed access constraints, exposing reuse opportunities that are currently rejected before interval compatibility is even considered.

A second direction concerns the exact planner. The current branch-and-bound implementation is intentionally simple: it explores compatible assignments recursively and prunes only when the partial allocation already exceeds the best complete solution. This makes the method suitable for small graphs, but leaves substantial room for stronger pruning. Future work could introduce lower-bound estimates derived from peak live memory, remaining tensor sizes, or type-specific pressure windows, allowing larger graphs to benefit from exact search without relying exclusively on the fixed 25-node threshold. The same extension could also make `-Dforce_bnb` safer for diagnostic use on medium-sized models.

The heuristic planner could also be extended beyond a single selected ordering. Since the evaluation shows that no `-Dstatic_planning` mode dominates across all models, a later version could run several ordering strategies during code generation and retain the allocation with the smallest total footprint. This multi-strategy mode would preserve the simplicity of the existing best-fit placement logic while removing the need for the user to identify the best ordering manually through `compare_static_codegen_flags.py`.

Finally, two modelling choices deserve refinement. The interval-overlap rule currently treats intervals that meet at one endpoint as conflicting; adopting half-open lifetime intervals would allow tensors whose lifetimes only touch at a boundary to reuse the same backing buffer when the producer-consumer ordering makes that reuse safe. In parallel, graph-output lifetimes should be represented explicitly instead of relying on the current `producer_step + 1` fallback. Handling outputs as first-class live ranges would make the lifetime model more accurate and reduce the number of conservative assumptions embedded in the planner.

6. NEW CI/CD

6.1 Motivation and Prior State

The Z-ANT continuous-integration pipeline had not been updated for seven months. The refactoring of Z-

ANT modules in the `feature` branch rendered all three workflows—`zig-tests.yml`, `zig-codegen-tests.yml`, and `zig-benchmarks.yml`—non-functional: build invocations referenced deprecated command interfaces, and the absence of a manual-dispatch trigger made it impossible to execute any workflow without committing directly to a protected branch. Furthermore, the environment setup appeared as a repeated, multi-step sequence inside every job, making the code hardly readable, difficult to maintain, and not easy to extend.

It had been a while since Z-ANT’s last stable release, and to make a new one a robust and comprehensive CI/CD is needed.

6.2 Setup fix and infrastructure refactoring

Since the previous zig setup was deprecated, a new zig setup was searched and implemented. Then, two *composite actions* [4, 5, 6, 7]—`setup-python` and `setup-zig`—consolidate duplicate logic into single reusable units stored under `.github/actions/`. Each action exposes default input values, reducing per-job environment configuration to a single invocation:

```
1 - name: Setup Zig
2   uses: ../github/actions/setup-zig
```

Listing 4. Invocation of the `setup-zig` composite action inside a job step.

Any future change to the setup procedure propagates automatically to all dependent jobs from a single definition file, granting better maintainability and easier changes in the event of action deprecation, as it happened with the previous action.

A `workflow_dispatch` event trigger is added to each workflow, allowing on-demand execution from the *GitHub Actions* web interface without requiring a push on critical or stable branches:

```
1 on:
2   workflow_dispatch:
```

Listing 5. `workflow_dispatch` trigger enabling manual workflow execution.

6.3 Coverage Expansion and Robustness

Between the several changes in the matrix strategy in all workflows, the most substantial coverage improvement is the extension of the matrix strategies to the two primary job groups. The `test-cross-platform` job exercises three target triples across two optimization levels, producing six configurations per run; `test-lib-generation` combines three independent boolean or enumerated axes, yielding eight library-build configurations per model:

```
1 matrix:
2   target: [x86_64-linux, aarch64-linux, x86_64-
3     windows]
3   optimize: [Debug, ReleaseFast]
```

Listing 6. Matrix for the cross-platform compilation job.

```
1 matrix:
2   model: [mnist-8]
3   comm: [true, false]
4   enable_user_tests: [true, false]
```

```
5   dynamic: ["", "-Ddynamic"]
```

Listing 7. Matrix for the library-generation job.

The expansion approximately doubles the total job count, increasing the flag combinations tested, as illustrated by the workflow graph comparison in fig. 4.



Figure 4. Codegen workflow before and after the CI/CD redesign. The original flat serial graph (left) is replaced by a matrix-expanded structure with correct dependency scoping and independent parallel cells (right).

In the original pipeline, the `needs: setup-models` field was declared on every codegen job regardless of whether the job consumed the downloaded model artefacts. Scoping the dependency to the jobs that genuinely require those artefacts removes the unnecessary synchronization point and eliminates redundant download overhead:

```
1 needs: setup-models
```

Listing 8. Over-applied dependency field, now scoped to jobs that require model artefacts.

6.4 Jobs’ outputs and debugging

The `fail-fast` option is set to `false` at the strategy level across all matrix jobs. Matrix cells are independent by construction; allowing a single failure to abort the remaining cells discarded the diagnostic output of otherwise unaffected configurations:

```
1 strategy:
2   fail-fast: false
```

Listing 9. `fail-fast` disabled to preserve independent matrix diagnostics.

Testing also surfaced four broken filesystem paths introduced by the preceding codebase refactoring: two references in `shape_thief.py` and two in `infer_shape.py` pointed to relocated modules. Each was corrected to match the reorganized layout. Residual build-flag references in the project documentation, rendered invalid by the CLI deprecation, were removed consequently.

Finally, the use of the Zig’s standard `--summary all` flag gives plenty of information on crashed workflows. That combined with GitHub’s actions history, storing every command output, makes logging easy to access and therefore aiding debugging.

7. FUTURE DEVELOPMENTS

7.1 CMSIS-NN as a Hardware Acceleration Target

CMSIS-NN is ARM’s open-source library of hand-optimized neural-network kernels for the Cortex-M micro-

controller family [8]. On Cortex-M4 and Cortex-M7 cores it exploits SIMD and DSP instruction-level parallelism; on Cortex-M55 and Cortex-M85 it additionally leverages the *Helium* M-Profile Vector Extension for vectorized integer arithmetic. The library targets the computationally dominant operations in quantized convolutional inference—convolution, depthwise convolution, pooling, and fully connected layers—and provides the ARM-standard kernel optimization layer for TinyML workloads on this hardware class.

Since Z-ANT deploys quantized ONNX models to Cortex-M targets and generates bare-metal C/Zig inference libraries, CMSIS-NN represents a direct acceleration path for the most compute-intensive operator in those workloads: quantized convolution. Integrating CMSIS-NN would allow Z-ANT to replace its native `QLinearConv` kernel with ARM’s hardware-optimized equivalent on compatible targets, while retaining the native implementation as a fallback for all other hardware. CMSIS-NN integration is therefore the most natural next step for Z-ANT, directly targeting the hardware capabilities of the embedded platforms for which the toolchain generates code.

7.2 Prior Integration and its Observed Limitations

An earlier version of Z-ANT received a CMSIS-NN backend targeting `QLinearConv`, developed by Del Grosso, Severino, and Scarabelli [2] on a Cortex-M4 platform (Nucleo-64 / STM32L476RGT6 at 80 MHz) and a Cortex-M7 platform (Arduino Nicla Vision / STM32H747XI at 480 MHz). The approach compiled CMSIS-NN C sources within the unified Zig build, selected the CMSIS or native execution path at compile time via `comptime` dispatch, and mediated the tensor layout and quantization mismatches between the ONNX-native representation and the CMSIS-NN calling convention through a dedicated wrapper. The integration was functionally complete: the CMSIS-NN path produced numerically correct outputs for all tested configurations.

The latency benchmarks, however, did not confirm a uniform improvement across kernel geometries. On 3×3 convolutions both platforms favoured CMSIS-NN—by up to 17.6 % on the Cortex-M7 and 21.9 % on the Cortex-M4—but on 5×5 kernels the native Z-ANT implementation was consistently faster, by up to 26.5 % on the M7 and 34.9 % on the M4 [2]. Depthwise convolution produced the most pronounced divergence: for depthwise 5×5 kernels the native path ran 57.8 % faster on the Cortex-M7 and reached approximately $2.8\times$ the speed of the CMSIS-NN path on the Cortex-M4. These results are summarised in fig. 5.

The investigation identified two principal factors behind the underperformance [2]. For larger standard kernels, the leading hypothesis is CMSIS-NN’s use of the *im2col* transformation to map convolutions to matrix multiplications: while this increases arithmetic throughput on cache-rich processors, it substantially expands memory traffic and buffer usage, and on Cortex-M targets with limited or absent data caches the resulting pressure can cause cache thrashing that negates the benefit of the ac-

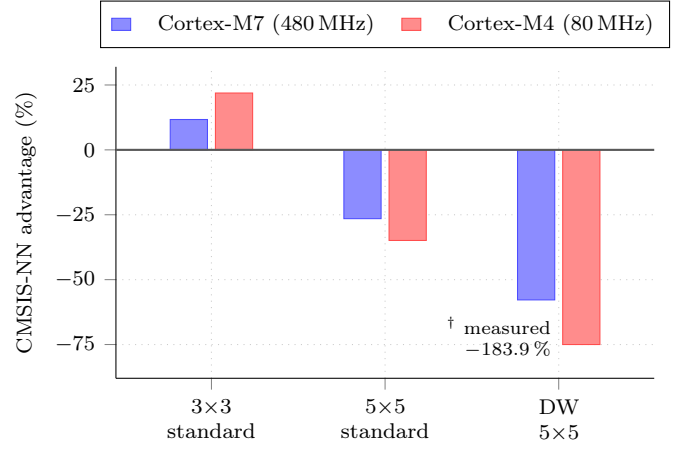


Figure 5. CMSIS-NN latency advantage over the native Z-ANT implementation for three representative kernel configurations on two Cortex-M platforms [2]. Positive values indicate CMSIS-NN is faster; negative values indicate the native path is faster. [†] The Cortex-M4 DW 5×5 bar is clamped at -75% ; the measured value is -183.9% .

celerated kernel. For depthwise convolution, the identified bottleneck is the mandatory runtime reordering of filter weights to satisfy the CMSIS-NN layout contract—a serial step preceding each kernel call that cannot be parallelised, imposing a fixed overhead before the DSP pipeline can be engaged.

7.3 Re-integration and Performance Objectives

A subsequent large-scale refactoring of the Z-ANT codebase, consolidated on the `feature` branch, reorganised the source tree into a three-module build graph—`zant_utils`, `IR_zant`, and `codegen`—eliminating circular module dependencies that had accumulated in the prior architecture and relocating the `QLinearConv` operator path into `IR_zant` under a centralised `qlinear_conv_dispatch` function. The `src/Core/Tensor/Cmsis/` subtree on which the prior integration depended was removed in this process, rendering the existing implementation structurally incompatible and necessitating a fresh re-implementation, whose current state is mentioned in section 7.4.

Once re-integration is complete, the primary objective is experimental validation on physical Cortex-M hardware and a targeted effort to address the performance shortfalls identified in section 7.2. The prior benchmarks showed that CMSIS-NN delivers a reliable latency advantage only for specific kernel configurations; the re-implemented backend must therefore be evaluated not only for correctness but for net benefit—establishing on which kernel classes and target cores CMSIS-NN acceleration yields a measurable improvement over the native Z-ANT implementation, and where the native path remains preferable.

7.4 Current Re-integration Status

A partial re-integration has already been initiated on the `feature` branch. The current implementation provides build-flag infrastructure (`-Denable_CMSIS` and `-Dforce_CMSIS`, with string-based Cortex-M CPU detection via `-Dcpu`), a shared CMSIS helper package under

`IR_zant` comprising a compile-time gate module, layout helpers for activation transposition and filter weight re-ordering, and quantization helpers for signed activation conversion and per-channel requantization parameters. A `QLinearConv` CMSIS bridge exposes an active NCHW entry point alongside a forward-compatible NHWC entry point; `qlinear_conv_dispatch` now attempts the CMSIS path first and falls back to the embedded implementation when the bridge rejects a configuration or CMSIS is inactive.

Three gaps remain before the experimental validation described in section 7.3 can be pursued. CMSIS-NN source files are not yet vendored under `third_party/`, preventing the CMSIS-enabled build path from being exercised without manual local preparation. Grouped convolution and depthwise convolution are not yet supported by the bridge—both fall back to the embedded implementation—and no benchmarks have been run on the new implementation, leaving the performance conclusions of section 7.2 as the only available latency data.

8. EVALUATION AND CONCLUSIONS

This paper presented a set of contributions to Z-ANT aimed at making the generated inference pipeline more reliable, memory-efficient, and maintainable for embedded deployment. The common direction across these changes is the same design principle that motivates the toolchain as a whole: decisions that are costly or unsafe on microcontroller-class targets should be resolved at compile time rather than deferred to runtime.

At the generated-code level, the positional error-reporting mechanism improves diagnosability by replacing the previous uniform `-1` return value with operator-indexed failure codes inside `predict()`. The corrections to `split` restore consistency with the ONNX type contract and with preallocated-output semantics, while the implementation of `abs` extends the operator library with both normal and lean execution paths. These changes are local, but they strengthen the correctness of the generated libraries and remove failure modes that would otherwise appear only during full inference execution.

The main technical contribution is the redesign of static memory planning. The v1 planner replaces the local greedy behaviour of v0 with an offline lifetime-aware interval-packing model, producing substantial allocation reductions on four of the five evaluated graphs. The `resnet50` result also clarifies the limitation of this approach: when residual connections force large activations to remain live across convergence points, buffer reassignment alone cannot reduce the footprint, and further gains require graph-level transformations rather than only a different packing order.

The CI/CD redesign restores the automation needed to preserve these properties as the codebase evolves. Repaired workflows, reusable setup actions, manual-dispatch support, matrix-based testing, and corrected filesystem paths make the project less vulnerable to silent regressions across target, optimization, testing, and memory-planning

configurations.

Finally, CMSIS-NN remains the principal future path for hardware acceleration. The previous integration showed that numerical correctness is not sufficient: larger kernels and depthwise configurations can lose performance because of memory traffic, buffer pressure, and layout-adaptation overhead. The next integration should therefore treat CMSIS-NN as a selective backend, enabled only where physical-hardware benchmarks demonstrate an advantage over the native Z-ANT implementation. Overall, the completed work strengthens Z-ANT’s compile-time deployment model, while leaving a concrete and well-scoped path toward further memory optimization and target-specific acceleration.

REFERENCES

- [1] Mirko Calvi. Zant: Lightweight neural network inference engine for microcontrollers. Master’s thesis, Politecnico di Milano, 2025. Available at <https://www.politesi.polimi.it/handle/10589/247070>.
- [2] Vincenzo Del Grosso, Emanuele Severino, and Riccardo Scarabelli. Optimizing edge AI: Integration of CMSIS-NN kernels into the ZANT cross-compilation framework. Technical report, Politecnico di Milano, DEIB, 2025.
- [3] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco, 1979.
- [4] GitHub. Continuous deployment. <https://docs.github.com/en/actions/get-started/continuous-deployment>, 2026. GitHub Actions Documentation, Get Started. Accessed: 2026-06-11.
- [5] GitHub. Continuous integration. <https://docs.github.com/en/actions/get-started/continuous-integration>, 2026. GitHub Actions Documentation, Get Started. Accessed: 2026-06-11.
- [6] GitHub. Creating a composite action. <https://docs.github.com/en/actions/tutorials/create-actions/create-a-composite-action>, 2026. GitHub Actions Documentation, Tutorials: Create Actions. Accessed: 2026-06-11.
- [7] GitHub. Reuse workflows. <https://docs.github.com/en/actions/how-tos/reuse-automations/reuse-workflows>, 2026. GitHub Actions Documentation, How-tos: Reuse Automations. Accessed: 2026-06-11.
- [8] Liangzhen Lai, Naveen Suda, and Vikas Chandra. CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs. *arXiv preprint arXiv:1801.06601*, 2018.
- [9] Z-Ant Contributors. Z-Ant: An open-source embedded inference framework. <https://github.com/ZantFoundation/Z-Ant>, 2024–2026. Accessed: June 2026.